LANDMARK METROPOLITAN UNIVERSITY INSTITUTE
THE PRIDE OF AFRICA

DATA STRUCTURES AND LOOPS

SE309 – Python Programming with Project

Precious Oben, 2025/2026

DATA STRUCTURES

Data structures are ways to organize, store, and manage data in a computer so that it can be accessed and modified efficiently. They provide a means to handle large amounts of data in an organized manner, which is essential for developing robust software applications.

Uses of Data Structures:

- **Organizing data:** They help in systematically arranging data to make it easier to work with.
- **Improving efficiency:** Efficient data structures can enhance the performance of algorithms, reducing the time and space complexity.
- **Facilitating data manipulation:** They allow easy insertion, deletion, retrieval and updating of data.
- **Solving complex problems:** Many algorithms are built upon specific data structures to optimize problem-solving.

DATA STRUCTURES

There are four built-in data structures in Python: Lists, Tuples, Dictionaries and Sets.

I – LISTS

Lists are used to store multiple items in a single variable. They are a mutable and ordered collection of items. This means items can be removed, added and changed after a list has been created and the items have a defined order which cannot be changed.

The general syntax for creating a list is:

mylist = [“abc”, “def”, “ghi”] , using square braces.

Lists can hold items of different data types and this values can be duplicates for example:

```
thislist = [“apple”, 1, True]
```

MANIPULATING LISTS

Accessing Lists

List items can be accessed through indexing.
The first item in a list has an index [0], the second [1] and so on.

Example

```
shoppingcart = [“books”, “pens”, “buckets”]
print(shoppingcart[1])
```

The example above should return pens as result.

List can also be indexed from the end through negative indexing. From the example above if we printed [-1] the result will be buckets and [-2] would be pens.

An index range can be used as well to access list items by specifying where to start and where to end.

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[2:5])  #this should return the third, fourth and fifth item
```

MANIPULATING LISTS

Modifying Values

To change or modify the value of a specific item in a list, refer to the index number:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
thislist[1] = “pineapple”
```

TRY

```
thislist[1:3] = [“strawberry", "watermelon"]
```

LIST METHODS

- append() – Adds elements to the end of a list
- copy() – returns a copy of the list
- count() – returns the number of elements with the specified value
- insert() – Adds an element at the specified position
- pop() – Removes an element at a specified position
- remove() – Removes the item with the specified value
- reverse() – Reverses the order of the list
- sort() – sorts the list
- extend()- Adds the elements of a list to the end of another list.

LIST COMPREHENSION

List comprehension is a shorter and efficient way to create lists by performing operations on existing iterables like lists, tuples or strings in a single line of code.

It allows you to create a new list by specifying an expression followed by a for loop and optionally an if condition to filter the items.

Basic Structure:

new_list = [expression for item in iterable]

Example: Creating a new list with the squares of numbers from 1 to 5

```
numbers = [1, 2, 3, 4, 5]
squares = [x**2 for x in numbers]
print(squares)
```

TUPLES

Tuples are like lists, they are also used to store multiple items in a single variable.

The general syntax for creating tuples is:

newtuple = (“ball”, “crayon”, “ruler”)

Tuple items are ordered, allow duplicate values and are unchangeable once created.
Tuples can contain different data types like lists, however, they cannot be manipulated like them because they are unchangeable.

Tuples can be accessed through indexing like lists. Note, the first item has index 0.

To manipulate tuple values, the tuples would have to be converted to lists first using the list() method, then later reconverted to tuples after the changes have been made using the tuple() method.

```
mytuple = (“one”, two”)
newlist = list(mytuple)
newlist[1] = “three”
```

DICTIONARIES

A dictionary is a collection of related data pairs (Key: value pairs). This can be used In a case where we want to store studentIDs and Student Names.

Dictionaries are written with curly brackets and have keys and values:

```
newdict ={
   “name” : “John”
   “age”   :  10
}
```

Dictionaries have defined orders which will not change. Their values are changeable but duplicates are not allowed. They can hold values of different data types including lists.

ACCESSING DICTIONARIES

Items of a dictionary can be accessed by referring to its key name inside square brackets or using the **.get()** method.

```
newdict[“name”]
newdict.get(“name”)
```

DICTIONARY METHODS

- get() – returns the value of a specified key
- pop() – removes the element with the specified key
- popitem() – removes the last inserted key – value pair
- update() – updates the dictionary with key – value pairs
- clear() – removes all elements from the dictionary
- values() – returns a list of all the values in the dictionary
- keys() – returns a list of the dictionary's keys

SETS

Sets are used to store many items in one variables. However, unlike lists, the items are unchangeable, unordered and cannot be indexed. The items can be removed, new items added and duplicates are not allowed.

Sets are declared with curly brackets.

myset = {“one”, False. 1}

Sets can hold values of different data types. Note: True and 1 are considered to be same value and the same goes for False and 0.

ACCESSING SETS

Sets cannot be accessed through indexing but a loop can be used to search if a particular value is present.

```
myset = {“one”, False, 1}
For x in myset:
    print(x)
```

SETS

Adding Items

- To add an item to a set, use the add() method.
- To add items from another set, use the update () method. The object in the update () method doesn't have to be a set. It can be any iterable object such as tuples, lists and dictionaries.

Removing Items

- Items can be removed from sets using the remove() , discard() and pop() methods.
- The clear() method empties the set.

LOOPS

Loops are a fundamental concept that allow blocks of code to be executed multiple times without writing them repeatedly. They are especially useful when performing repetitive tasks.

Why Use Loops?

- Automation: Loops help automate repetitive tasks.
- Efficiency: Performing bulk operations with minimal code.
- Data Processing: Iterating through datasets like lists or files.

Python provides two types of loops:

- The **For** Loop &
- The **While** Loop

THE WHILE LOOP

The while loop executes a block of code as long as a specified condition is TRUE. It is useful when the number of iterations is not known before hand.

Syntax:

```
while condition:
    #code for execution
```

Example:

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

THE FOR LOOP

The For loop is used to iterate over sequences such as lists, tuples, strings or ranges. It goes through each item in the sequence and executes the block of code inside the loop for each item.

Syntax:

```
for variable in sequence:
    # code to execute
```

Example:

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

In the example above, the for loop iterates each item in the list and prints it.

THE FOR LOOP

Looping Through Strings

Strings are iterable objects as they contain a sequence of characters.

```
for x in “gold":
  print(x)
```

Looping through Ranges

Ranges are used to loop through a set of code a specified number of times for example;

```
for x in range(5):
  print(x)
```

The default starting value of the range function is 0, however, it is possible to specify the start and end values e,g range(1,5)

BREAK & CONTINUE STATEMENTS

Python also provides control statements to alter the flow of loops:

- Break: Exits the loop prematurely when a certain condition is met.
- Continue: Skips the rest of the code inside the loop for the current iteration and moves to the next iteration.

1.

```
i = 1
while i < 10:
    print(i)
    break  # Exits the loop immediately
    i += 1
```

2.

```
for number in range(5):
    if number == 3:
        continue
    print(number)
```

ELSE KEYWORDS IN LOOPS

The Else keyword can be used in both for and while loops. The else block only runs if the loop completes normally. (i.e it does not encounter a break statement).

Syntax of Else in Loops:

```
for item in iterable:
    # Loop body
else:
    # Executes if the loop is not terminated by a break
```

Example:

```
for i in range(3):
    print(i)
else:
    print("Loop finished without break.")
```

EXERCISES

1. Create a program that interacts with the user to add items to a list and then search for a specific item from that list. If the item is found, it prints an acknowledgement, else “Target was not found”.
2. Create a program that allows users to manage their tasks by adding, viewing and searching for tasks in a to – do list.
3. Create a simple Library Management System where users can borrow, return, and view borrowed books. The system should provide a menu with options to borrow a book (add it to a list), return a book (remove it from the list), and view all borrowed books (display the list). Users should be able to interact with the system by entering the name of the book they want to borrow or return, and the program should handle invalid inputs such as attempting to return a book not borrowed. The list of borrowed books should be displayed with an index, and the program should continue running until the user chooses to exit. This exercise tests your understanding of lists, loops, and conditionals in Python.